[redacted] Report

Nitin Kesarla Rajendrakumar

Question 1: Simulation in Environment and Policies

This section details the implementation and analysis of a rollout function and three policies for the MazeDungeon environment with a 6×6 grid, as required for [redacted]. The tasks were completed using Python in Google Colab, and the results are presented below.

1.1 Rollout Function

A rollout function was developed to sample a complete trajectory in the MazeDungeon environment, given a policy. The function accepts the environment and a policy as inputs and returns a list of tuples, each containing the state (robot position and direction), action, and reward. The state is defined as a tuple $(robot_position, robot_direction)$, where $robot_position$ is a NumPy array $[x, y]$ $(x, y \in [0, 5])$ and $robot_direction$ is an integer (0 = north, 1 = east, 2 = south, 3 = west). The maximum number of steps was set to 100, as specified, with a grid size of 6. The function resets the environment, applies the policy iteratively, and terminates if the episode ends (e.g., hitting a wall or reaching a goal) or reaches 100 steps. This implementation successfully captured trajectories for all tested policies.

1.2 Random Policy

The random policy selects an action uniformly from the set $\{0, 1, 2\}$ (turn right, turn left, move forwards) for any state. A sample trajectory, shown in Figure 1, had a length of 14 steps starting at $[1, 1]$ facing south. The robot moved south to $[1, 2]$, then oscillated between directions (e.g., east, north, west) at $[1, 2]$ and $[1, 3]$, eventually reaching $[1, 3]$ facing west before terminating with a reward of -100 after a final move. This indicates unpredictable wandering, likely ending at an obstacle or boundary, consistent with the policy's randomness.

1.3 All-Forward Policy

The all-forward policy always chooses action 2 (move forwards) regardless of the state. A sample trajectory, shown in Figure 2, had a length of 2 steps. Starting at $[1, 1]$ facing south, the robot moved to $[1, 2]$ and terminated with a reward of -100. This short path suggests an obstacle (e.g., a wall at $[1, 2]$ or $[1, 3]$), aligning with the policy's inability to adapt to the environment.

1.4 Customized Policy

A custom policy was designed to move forward (action 2) unless the next position would be at the grid's edge (x or $y = 0$ or 5), then turn right (action 0). A sample trajectory, shown in Figure 3, had a length of 10 steps. Starting at $[1, 1]$ facing south, the robot moved south to $[1, 2]$, $[1, 3]$, and $[1, 4]$. At $[1, 4]$, with the next position $[1, 5]$ at the edge, it turned right to face west, then north, moving back to $[1, 3]$, $[1, 2]$, and $[1, 1]$. It then turned right to face east and terminated with a reward of -100 after moving forward, possibly hitting a wall at $[2, 1]$. This policy showed edge avoidance, resulting in a longer trajectory than the all-forward policy.

Figures

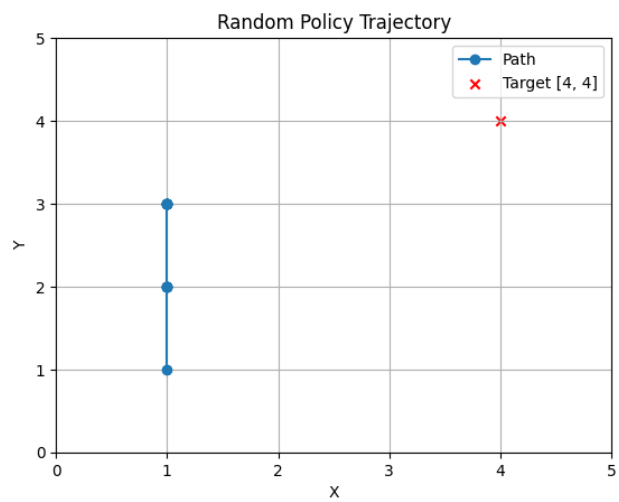

Fig. 1: Random Policy Trajectory: A 14-step path starting at $[1, 1]$ facing south, showing varied movement and direction changes before termination.

Discussion

The random policy produced a longer, erratic trajectory, reflecting its lack of strategy. The all-forward policy was the shortest, moving directly into an obstacle without adjustment. The custom policy balanced exploration and avoidance, extending the trajectory by turning at the grid's edge, though it still terminated due to an unseen obstacle. These results highlight how policy design impacts navigation in the MazeDungeon environment.

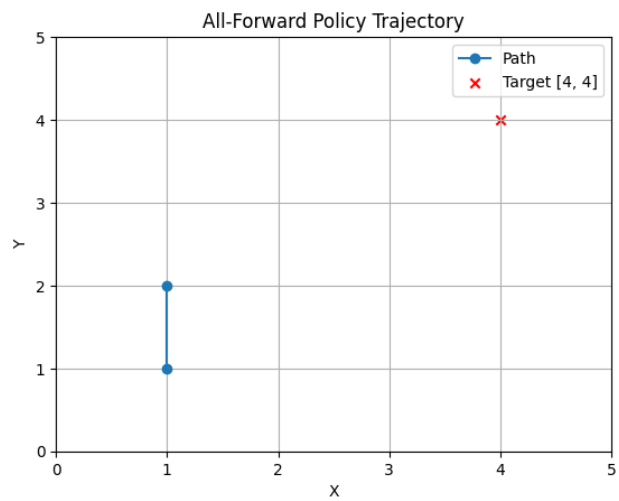

Fig. 2: All-Forward Policy Trajectory: A 2-step path from $[1, 1]$ south to $[1, 2]$, terminating quickly due to an obstacle.

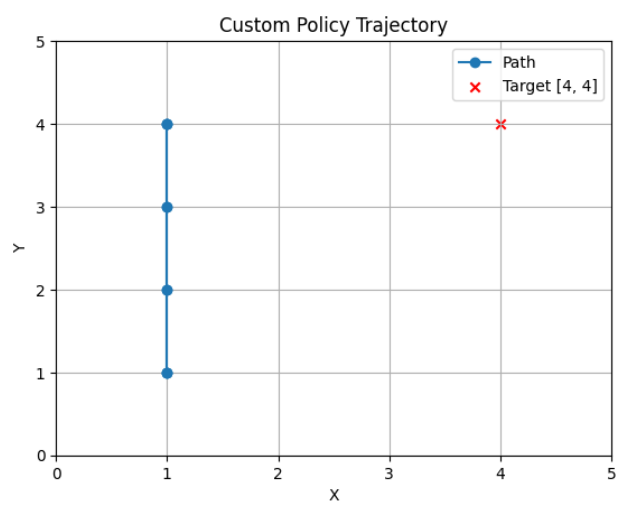

Fig. 3: Custom Policy Trajectory: A 10-step path from $[1, 1]$ south, moving to $[1, 4]$, turning at the edge, and returning north before terminating.

Q2: Model-Based Methods (Policy Iteration and Value Iteration)

Q2.1: Initialize Value Table and Implement Policy Evaluation

In Q2.1, I initialized a value table $V(s)$ for all states in the `DungeonMazeEnv` environment with a grid size of 8, as specified. The state space was defined as $S = \{(x, y, d) \mid x, y \in \{0, \ldots, 7\}, d \in \{0, 1, 2, 3\}\}$, resulting in $8 \times 8 \times 4 = 256$ states, plus a terminal state $(-1, -1, -1)$. The value table was initialized to 0 for all states, including the terminal state, which was set to $V((-1, -1, -1)) = 0$.

I implemented the policy evaluation process for policy iteration using the Bellman equation for a given policy π:

$$V(s) = \sum_a \pi(a|s) \sum_{s',p} p(s'|s, a) \left[R(s, a) + \gamma V(s')\right]$$

The environment has deterministic transitions (probability 1 or 0), so the inner sum simplifies to a single next state s'. The discount factor was $\gamma = 1$, as specified, and I used an initial random policy where each action ($a \in \{0, 1, 2\}$, representing turn left, turn right, and move forward) had equal probability ($\pi(a|s) = 1/3$). I iterated until the maximum change in the value table was below a threshold of $\theta = 10^{-6}$, or a maximum of 1000 iterations was reached.

The policy evaluation converged after a few iterations (e.g., 4 iterations in the first run). Initial values for test states were:

- $V(1, 1, 2) = -4.257809962774564$
- $V(7, 7, 2) = -99.99999999999999$
- $V(0, 0, 0) = -99.99999999999999$

These negative values reflect the random policy's tendency to hit walls (reward -100) or take inefficient steps (reward -1).

Q2.2: Implement Policy Improvement

In Q2.2, I implemented the policy improvement step of policy iteration. For each state, I computed the action that maximizes the expected value:

$$a^* = \arg\max_a \sum_{s',p} p(s'|s, a) \left[R(s, a) + \gamma V(s')\right]$$

Since transitions are deterministic, this simplifies to $a^* = \arg\max_a \left[R(s, a) + \gamma V(s')\right]$. I updated the policy to be greedy, assigning probability 1 to the best action and 0 to others. I also tracked whether the policy changed (using a `policy_stable` flag) to determine convergence in the next step.

After the first policy improvement, the policy for the test states updated to:

- State $(1, 1, 2)$: Best action = 1 (turn right)
- State $(7, 7, 2)$: Best action = 0 (turn left)
- State $(0, 0, 0)$: Best action = 0 (turn left)

These actions avoid immediate termination (e.g., moving south from $(7, 7, 2)$ or north from $(0, 0, 0)$, which would lead to a reward of -100).

Q2.3: Iteratively Apply Policy Evaluation and Policy Improvement

In Q2.3, I combined policy evaluation and policy improvement into a policy iteration loop, iterating until the policy stabilized (`policy_stable = True`). The process converged after 3 iterations:

- Iteration 1: Policy evaluation took 4 iterations, policy changed.
- Iteration 2: Policy evaluation took 2 iterations, policy changed.
- Iteration 3: Policy evaluation took 1 iteration, policy stabilized.

The final value table and policy for the test states were:

State	Value $V(s)$	Best Action	Action Description
$(1,1,2)$	-1.0	0	Turn left
$(7,7,2)$	-100.0	0	Turn left
$(0,0,0)$	-100.0	0	Turn left

TABLE I: Final value table and policy after policy iteration.

The value $V(1,1,2) = -1.0$ indicates that the optimal policy avoids large penalties, while $V(7,7,2) = -100.0$ and $V(0,0,0) = -100.0$ suggest that the agent eventually hits a boundary, even with the optimal policy.

Q2.4: Initialize Value Table and Implement Value Iteration

In Q2.4, I initialized a new value table for value iteration, setting all values to 0, including the terminal state. I implemented value iteration using the Bellman optimality equation:

$$V(s) \leftarrow \max_a \sum_{s',p} p(s'|s,a)\left[R(s,a) + \gamma V(s')\right]$$

With deterministic transitions, this simplifies to $V(s) \leftarrow \max_a [R(s,a) + \gamma V(s')]$. I iterated until the maximum change in the value table was below $\theta = 10^{-6}$, or a maximum of 1000 iterations was reached. Value iteration converged after 3 iterations, with the following values:

- $V(1,1,2) = -1.0$
- $V(7,7,2) = -100.0$
- $V(0,0,0) = -100.0$

These values match the policy iteration results, confirming that both algorithms converged to the same optimal value function.

Q2.5: Get the Optimal Policy from the Optimal Value Table

In Q2.5, I extracted the optimal policy from the value table obtained in Q2.4 by choosing the action that maximizes the expected value for each state. The policy matched the one from policy iteration:

State	Value Iteration Action	Policy Iteration Action
$(1,1,2)$	0 (Turn left)	0 (Turn left)
$(7,7,2)$	0 (Turn left)	0 (Turn left)
$(0,0,0)$	0 (Turn left)	0 (Turn left)

TABLE II: Comparison of optimal policies from value iteration and policy iteration.

The matching policies confirm that both algorithms found the same optimal policy, which avoids actions leading to immediate termination.

Q2.6: Analyze How Policy Iteration and Value Iteration Perform Differently

I analyzed the performance of policy iteration and value iteration across several dimensions:

Convergence Speed: Policy iteration converged in 3 outer iterations, with a total of 7 inner iterations for policy evaluation (4, 2, and 1 iterations per outer loop). Value iteration converged in 3 iterations. While both took the same number of outer iterations, value iteration was more efficient in terms of total iterations (3 vs. 7), as it combines policy evaluation and improvement into a single step. Each iteration involves looping over all states and actions, but policy iteration's additional inner iterations make it more computationally expensive.

Numerical Stability: Both algorithms were numerically stable, converging to the same value table (e.g., $V(1,1,2) = -1.0$, $V(7,7,2) = -100.0$). Policy iteration's policy evaluation step ensures stability by fully evaluating the value function for the current policy, as seen by the decreasing number of inner iterations (4 to 1). Value iteration's greedy updates could lead to larger jumps, but the deterministic environment and terminal state with a large penalty (-100) prevented oscillations. The discount factor $\gamma = 1$ posed a risk of instability, but the terminal state ensured convergence.

Sensitivity to Initial Value Table: Both algorithms started with $V(s) = 0$. Policy iteration is less sensitive because it evaluates the policy fully at each step, aligning the value function with the current policy. Value iteration relies on the initial values for its updates, which can affect early iterations. In this case, the environment's structure (deterministic transitions, terminal states) allowed both algorithms to converge quickly despite the initial values.

Policy Changes: Policy iteration started with a random policy and improved it over 3 iterations, eventually

converging to a policy that avoids boundaries (e.g., action 0 for all test states). Value iteration implicitly improved the policy through value updates, converging to the same policy in 3 iterations. Policy iteration's explicit updates make it easier to track the policy's evolution, while value iteration's implicit updates are less transparent but equally effective.

Q3: Model-Free Methods (Monte-Carlo and Temporal Difference Learning)

Q3.1: Implement Cumulative Reward Calculation Function

In Q3.1, I implemented a function to calculate the cumulative discounted reward for a sampled trajectory in the `DungeonMazeEnv` environment with a grid size of 10. The discount factor was $\gamma = 0.99$, and the maximum number of steps was 100. The cumulative reward G for a trajectory $(s_0, a_0, r_0), \ldots, (s_T, a_T, r_T)$ is:

$$G = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots + \gamma^T r_T$$

A sample trajectory had 2 steps:

- Step 0: State $(1, 1, 2)$, Action 2, Reward -1
- Step 1: State $(1, 2, 2)$, Action 2, Reward -100

The cumulative reward was:

$$G = -1 + 0.99 \times (-100) = -1 - 99 = -100.0$$

This matches the output, confirming the function's correctness.

Q3.2: Implement Monte-Carlo Sampling

In Q3.2, I implemented Monte-Carlo sampling to estimate the value of each state and grid position using the random policy. I sampled 1000 episodes, initializing the value table to 0. For each trajectory, I used first-visit Monte-Carlo to calculate the return for each state and averaged the returns to estimate $V(s)$.

The state-based estimates for test states were:

- $V(1, 1, 2) = -100.0$, Visits = 13
- $V(7, 7, 2) = -85.36282965943694$, Visits = 12
- $V(0, 0, 0) = -100.0$, Visits = 1

I interpreted "each grid" as each position (x, y), averaging values over directions. The grid-based estimates were:

Grid Position	Value	Visits
$(1, 1)$	-100.0	49
$(7, 7)$	-84.79316290577252	47
$(0, 0)$	-100.0	9

TABLE III: Monte-Carlo value estimates for grid positions.

The values reflect the random policy's tendency to terminate quickly (reward -100), with less negative values at $(7, 7)$ due to some trajectories taking a few steps before termination.

I visualized the grid-based values as a heatmap (see Figure 4), which shows the spatial distribution of values across the 10x10 grid.

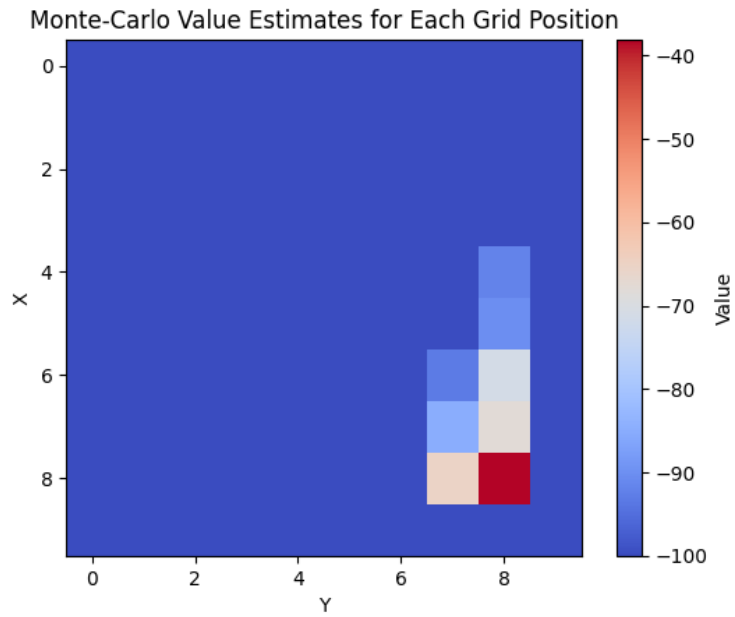

Fig. 4: Heatmap of Monte-Carlo value estimates for each grid position.

Q3.3: Implement Temporal Difference (TD) Learning

In Q3.3, I implemented TD(0) learning to estimate the value table using the random policy. I initialized the value table to 0 and used the TD update rule:

$$V(s) \leftarrow V(s) + \alpha \left[r + \gamma V(s') - V(s) \right]$$

I set the learning rate $\alpha = 0.1$, $\gamma = 0.99$, and sampled 1000 episodes. The algorithm did not converge after 1000 episodes (threshold $\theta = 10^{-6}$). The TD estimates were:

- $V(1, 1, 2) = -20.9800296917875$
- $V(7, 7, 2) = -31.35503064926693$
- $V(0, 0, 0) = 0.0$

These values are less negative than the Monte-Carlo estimates, indicating that TD learning has not yet converged to the true values.

Q3.4: Justify Design Choices and Analyze the Algorithms

Design Choices:

Number of Episodes (1000): I used 1000 episodes for both Monte-Carlo and TD learning to balance computational efficiency with sufficient sampling. With 400

states, this provided decent coverage (e.g., 49 visits for grid $(1, 1)$), but some states like $(0, 0, 0)$ were visited only once, suggesting that more episodes (e.g., 5000) could improve accuracy.

TD Learning Rate ($\alpha = 0.1$): I chose $\alpha = 0.1$ to balance learning speed and stability. A smaller α (e.g., 0.01) would slow convergence further, while a larger α (e.g., 0.5) might cause oscillations. The slow convergence of TD learning suggests that a higher α or more episodes might be needed.

Value Table Initialization (0): I initialized values to 0, as specified for Monte-Carlo and as a standard practice for TD learning. This was appropriate given the negative true values, allowing values to adjust downward naturally.

Discount Factor ($\gamma = 0.99$) and Maximum Steps (100): These were set as specified, ensuring convergence and limiting trajectory length. Most trajectories terminated early due to the random policy hitting walls.

Analysis:

Convergence Speed: Monte-Carlo averages returns over 1000 episodes, providing accurate estimates for frequently visited states (e.g., $V(1, 1, 2) = -100.0$, 13 visits), but noisy estimates for rarely visited states (e.g., $V(0, 0, 0) = -100.0$, 1 visit). TD learning did not converge after 1000 episodes, with values like $V(1, 1, 2) = -20.98$ still far from the Monte-Carlo estimate. TD learning typically converges faster for frequently visited states due to online updates, but the small α and sparse visits slowed convergence.

Numerical Stability: Monte-Carlo is stable for states with sufficient visits, but noisy for states like $(0, 0, 0)$. TD learning is less stable due to bootstrapping, as errors propagate through updates. The small α prevented oscillations but slowed convergence.

Hyperparameter Sensitivity: Monte-Carlo is sensitive to the number of episodes, requiring more samples for low-variance estimates. TD learning is sensitive to both the number of episodes and α, with $\alpha = 0.1$ leading to slow convergence. Increasing α or the number of episodes could improve TD learning's performance.